

3. Chaos Mesh 2.0

1. Chaos Mesh: 엣지 컴퓨팅 시대의 필수 도구, 혼돈 엔지니어링을 쉽게 만들다

복잡해지는 클라우드 네이티브 환경에서 시스템의 안정성을 확보하기 위한 혼돈 엔지니어링을 Chaos Mesh가 어떻게 쉽고 효과적으로 만들고 있는지 알아본다.

1.1. 혼돈 엔지니어링과 Chaos Mesh의 탄생 배경

- 복잡하고 분산된 클라우드 네이티브 환경에서는 예측 불가능한 장애가 언제 어디서든 발생할 수 있다.
- 이러한 장애에 대비하고 시스템의 안정성을 높이기 위해, 의도적으로 시스템에 문제를 일으켜 테스트하는 '혼돈 엔지니어링'이 중요해지고 있다.
- 기존의 테스트 및 디버깅 방식으로는 복잡한 분산 시스템의 잠재적 문제를 파악하고 해결하기 어려웠다.
- Chaos Mesh는 이러한 문제를 해결하기 위해 탄생했으며, 처음에는 쿠버네티스 환경에서 잘 작동하는 장애 주입 및 측정 도구를 만드는 것이 목표였다.
- 오픈 소스로 공개된 이후 커뮤니티의 도움으로 웹 UI, 데이터 검증, 다양한 장애 주입 기능 등을 갖춘 강력한 플랫폼으로 발전했다.

1.2. Chaos Mesh의 아키텍처와 핵심 구성 요소

- Chaos Mesh는 모듈식이고 단순한 아키텍처를 가지며, 쿠버네티스 운영자 모드와 유사하게 CRD(Custom Resource Definition)를 기반으로 작동한다.
- **주요 구성 요소:**
 - **Chaos Dashboard:** 사용자가 혼돈 실험을 설계하고 관리할 수 있는 웹 UI이다.
 - **Chaos Controller Manager:** 혼돈 실험의 상태를 스케줄링하고 모니터링하는 핵심 컴포넌트이며, 네이티브 워크플로우 엔진을 포함한다.
 - **Chaos Runner:** 각 쿠버네티스 노드에 DaemonSet 형태로 배포되어 실제 장애를 주입하는 컴포넌트이다.
- **작동 방식:**
 - 사용자는 Dashboard나 `kubectl apply` 명령어를 통해 혼돈 실험을 정의한다.

- 정의된 실험은 쿠버네티스 API 서버로 제출되고, Chaos Controller Manager가 이를 받아 스케줄링 및 실행 요구사항을 설정한다.
- Chaos Runner는 Controller Manager로부터 요청을 받아 대상 파드의 네트워크 네임스페이스나 PID 네임스페이스에 접근하여 장애를 주입한다.
- 네트워크 장애의 경우 tc 규칙을 설정하고, 스트레스 테스트의 경우 stress-ng 프로그램을 실행하여 CPU나 메모리를 사용한다.

2. Chaos Mesh 2.0: 혼돈 엔지니어링을 더욱 쉽고 강력하게 만들다

Chaos Mesh 2.0은 사용자 경험 개선, 워크플로우 기능 강화, 새로운 장애 유형 추가, 그리고 쿠버네티스 외 환경 지원 확대를 통해 혼돈 엔지니어링을 한 단계 발전시킨다.

2.1. Chaos Mesh 2.0의 주요 개선 사항

- **사용자 경험 개선:**
 - **Dashboard 최적화:** 혼돈 실험 생성 단계를 1~2단계로 줄여 사용 편의성을 높였다.
 - **이벤트 기반 정보 제공:** 커뮤니티 이벤트 메커니즘을 활용하여 실험의 생성 단계, 진행 상황, 결과 등을 Dashboard에서 시각적으로 확인할 수 있다.
 - **KLS Control 도구:** 문제 해결 및 장애 복구를 위한 디버깅 도구를 제공한다.
- **스케줄링 기능 강화:**
 - **Schedule 객체 도입:** 쿠버네티스 CronJob과 유사하게 특정 시간이나 주기마다 혼돈 실험을 실행하도록 예약할 수 있다.
- **네이티브 워크플로우 엔진:**
 - 여러 장애를 조합하거나 순차적으로 실행하고, 중간에 애플리케이션 상태를 검사하는 등 복잡한 혼돈 실험 시나리오를 정의할 수 있다.
 - 워크플로우는 트리 구조처럼 시작점부터 각 템플릿을 순차적 또는 병렬적으로 실행한다.
- **새로운 장애 유형 추가:**
 - 스트레스(CPU, 메모리), DNS, JMX, AWS 관련 장애(인스턴스 중지/재시작, S3 디스크 문제 등), GCP 관련 장애 등을 지원한다.
 - 특히, StressChaos는 CPU 또는 메모리 과부하를 시뮬레이션하고, DNS Chaos는 DNS 실패 또는 잘못된 IP 반환 등을 시뮬레이션한다.

- **Cald 컴포넌트 도입:**

- 쿠버네티스 환경이 아닌 독립적인 가상 머신이나 물리적 노드에서도 장애를 주입할 수 있도록 지원한다.
- Cald와 Chaos Mesh를 함께 사용하여 쿠버네티스 및 비쿠버네티스 환경을 혼합한 실험도 가능하다.

2.2. 보안 및 향후 계획

- **보안 강화:**

- RBAC(Role-Based Access Control) 기반의 인증 메커니즘을 도입하여 사용자별 애플리케이션 접근 권한과 혼돈 실험 실행 권한을 연동한다.
- 이를 통해 사용자가 자신의 권한 범위 내에서만 혼돈 실험을 수행하도록 제한하여 오작동을 방지한다.

- **향후 계획:**

- **통합 관리 플랫폼:** 비쿠버네티스 및 KubeRay 환경까지 포함하는 통합 관리 플랫폼을 제공할 예정이다.
- **상태 검사 및 보고서 개선:** 더 간결하고 강력한 상태 검사 메커니즘과 상세한 보고서를 제공할 계획이다.
- **플러그인 확장성:** 사용자가 새로운 장애 유형을 직접 개발하고 공유할 수 있는 플러그인 방식과 허브를 구축할 예정이다.
- **생태계 통합:** ArgoCD, Jenkins 등 다양한 CI/CD 도구와의 통합을 강화하여 GitOps 및 CI/CD 파이프라인 내에서 Chaos Mesh를 쉽게 활용할 수 있도록 지원한다.

2.3. Chaos Mesh 설치 및 데모 시연

- **설치 방법:**

- 프로덕션 환경에서는 Helm을 사용하여 설치하는 것을 권장하며, 로컬 테스트 환경에서는 스크립트 설치도 가능하다.

- **데모 시연 (네트워크 지연 주입):**

- 'Webshow'라는 애플리케이션을 사용하여 네트워크 지연을 시뮬레이션하는 과정을 보여준다.
- Chaos Mesh Dashboard에서 'Network Chaos'를 선택하고, 10ms의 네트워크 지연을 설정한다.

- 대상 파드를 'default' 네임스페이스의 'webshow' 파드로 지정하고, 실험 이름을 'network delay'로 설정한다.
- 30초 동안 네트워크 지연을 적용한 후, Webshow 애플리케이션에서 지연 시간이 1ms에서 10ms로 증가하는 것을 확인한다.
- 실험이 완료되면 정상 상태로 복구되는 것을 볼 수 있다.
- Schedule 기능을 사용하면 특정 시간마다 반복적인 네트워크 장애를 발생시킬 수도 있다.

2.4. Chaos Mesh 커뮤니티와 참여 방법

- 혼돈 엔지니어링 분야는 아직 새롭고 빠르게 발전하고 있으며, Chaos Mesh 역시 젊은 프로젝트이다.
- **참여 방법:**
 - GitHub 저장소에 이슈를 제출하거나 PR을 통해 기여할 수 있다.
 - CNCF Slack의 'chaos-mesh' 채널에서 토론에 참여하거나 질문할 수 있다.
 - 공식 웹사이트의 문서, 튜토리얼, 블로그, YouTube 채널을 통해 정보를 얻고 학습할 수 있다.
 - 월간 커뮤니티 미팅에 참여하여 최신 정보를 얻고 다른 사용자와 교류할 수 있다.